Programming in Java – Assignment 2 (CO2): Strings and Collections

Name: RITHIKA R H

Register No.: 1972260029

GitHub repository: `CSA09-1972260029-Assignment2`

link: https://github.com/rithikarh07-hue/CSA09-1972260029-Assignment2.git

Measurement method (applies to Q1–Q7)

- Time: `System.nanoTime()` around a batch of operations, a warm-up first, then the average (or the best) of several repeated batches, divided by the number of operations.
- Memory: bytes allocated by the thread, read from `com.sun.management.ThreadMXBean.getThreadAllocatedBytes`. This is more reliable than "used heap before/after", which is disturbed by garbage collection.
- Operation counts (elements visited, objects created) are reported next to the times, because the assignment treats them as primary evidence.
- Limits: JIT compilation makes the smallest sizes noisy (those cells are marked); numbers are for trend and ratio, not absolute speed. Use JMH if you want publication-grade results.

Q1 – Building Text Under Load: String, StringBuilder or StringBuffer? (6 marks)

Prediction (before implementation)

`String` is immutable. In `s = s + field` every iteration creates a *new* String and copies all characters built so far. For a ticket of *n* fields of average length *L* characters, iteration *i* copies about *i·L* characters, so the total is L·n(n+1)/2 = **O(n²)** characters copied and about *n* intermediate Strings are created (garbage). `StringBuilder` appends into a growable `char`/`byte` array that doubles when full, so the total copying is **O(n·L)** and only one builder plus a few array re-allocations are created. `StringBuffer` does the same work but every method is `synchronized`, so I predicted a small extra cost.

Predicted behaviour: for one short ticket the three are about equal (the difference is negligible); for 10,000 tickets of 50 fields + is several times slower and allocates many times more; for a very long ticket + collapses quadratically; `StringBuffer` is a little slower than `StringBuilder` when only one thread uses it.

Workloads and implementation

Workload	Design reason
W1: 8 fields per ticket, 100,000 tickets (timed per ticket)	Short ticket – tests whether the choice matters at all
W2: 50 fields per ticket, 10,000 tickets	The contrasting workload suggested in the brief

Workload	Design reason
W3 (my own): 20,000 fields per ticket (a very long itinerary), 3 tickets	Pushes *n* high to expose the $O(n^2)$ term

```
static String plus(String[] f)  { String s = ""; for (String x : f) s = s + x + ";";
return s; }
static String sb(String[] f)    { StringBuilder b = new StringBuilder(); for (String
x : f) b.append(x).append(';'); return b.toString(); }
static String sbuf(String[] f)  { StringBuffer b = new StringBuffer();  for (String x
: f) b.append(x).append(';'); return b.toString(); }
static String join(String[] f)  { return String.join(";", f) + ";"; }   //
alternative concept
```

Measured results (average of repeated batches; allocation measured on one ticket)

Workload	Approach	Time per ticket	Bytes allocated per ticket
W1 (8 fields)	String +	270 ns	848
	StringBuilder	249 ns	512
	StringBuffer	449 ns	520
	String.join	230 ns	352
W2 (50 fields)	String +	1,900 ns	21,680
	StringBuilder	528 ns	3,248
	StringBuffer	482 ns	3,256
	String.join	352 ns	1,864
W3 (20,000 fields)	String +	305,576,009 ns (≈ 0.31 s)	4,057,670,080 (≈ 4.06 GB)
	StringBuilder	213,159 ns (≈ 0.21 ms)	1,597,736 (≈ 1.6 MB)
	StringBuffer	292,211 ns	1,597,744
	String.join	218,913 ns	915,664

Intermediate String objects (estimate): + creates about *n* intermediate Strings per ticket (8, 50 and 20,000 for W1, W2, W3) and copies ≈ $L \cdot n^2/2$ characters; the builder creates 1 builder object, about $\log_2(n \cdot L)$ internal array re-allocations and 1 final String. The measured allocation agrees: for W3 the + ticket allocated about 2,500 times more memory than the builder.

Prediction versus observation

- W1: the times overlap (the best batches were 96–193 ns; the averages are inside the JIT warm-up noise), so "the choice is negligible for a short ticket" is supported in time, although + already allocated 66% more bytes (848 vs 512).
- W2: + was about 3.6× slower and allocated about 6.7× more than StringBuilder – as predicted.

- W3: `+` was about 1,430× slower and allocated about 2,540× more – the quadratic term dominates.
- Not as predicted: StringBuffer was *not* measurably slower than StringBuilder when used by one thread (W2 even shows 482 vs 528 ns, inside the noise). A likely reason is that the JIT compiler makes uncontended locks very cheap or removes them; I did not verify this. The synchronisation cost matters under contention, not in this single-thread test.

Design decision for the multithreaded booking server

- **Per-ticket text: `StringBuilder`.** A ticket is built inside one thread and the builder is a local variable that never escapes, so no synchronisation is needed.
- **Shared booking log: not a plain `StringBuilder`.** I tested 8 threads, each appending 20,000 times to one shared builder (50 trials): `StringBuilder` gave a wrong final length or threw an exception in **7 of 50 trials** (on average 675,406 characters lost in the bad trials), while `StringBuffer` was correct in **50 of 50**. So the shared log should be a `StringBuffer`, or better a `ConcurrentLinkedQueue<String>` / a lock / a logging framework. Caveat: `StringBuffer` makes each *single* `append` atomic, so a log entry written with several appends can still interleave with another thread's entry; append one finished ticket string per call.

Alternative Concept and Complexity Analysis – `String.join`, `StringJoiner`, `String.format`

`String.join(";", fields)` first computes the total length, allocates one array of exactly that size and fills it once: one pass, **O(n·L)** time, no re-growth (measured 352 ns for W2 and only 915,664 B for W3, less than StringBuilder's 1.6 MB because there is no doubling slack). `StringJoiner` behaves like a builder with delimiters built in. `String.format` also runs in O(n·L) but parses a format string every call, so it is slower per field; I did not measure it, and it is best kept for a few formatted fields, not for a loop of thousands.

	Repeated +	StringBuilder	`String.join`
Time for *n* fields	Θ(L·n²) characters copied	O(n·L) amortised	O(n·L), single pass
Extra (garbage) space	Θ(L·n²) characters in total, though only O(n·L) is alive at one moment	O(n·L): capacity up to 2× the content, plus the final copy	O(n·L): exactly one array plus the result

Evidence: prediction, three workloads, four implementations, measured time and allocation, prediction-vs-observation, thread-safety evidence and recommendation.

Q2 – When Is ArrayList an Appropriate Engineering Choice? (4 marks)

Experiments

Three playlist sizes (10,000; 100,000; 1,000,000 songs). Every operation was repeated many times after a warm-up; "insert/remove" cells are the average of one insert plus one remove at that position so the list size stays the same. Values are **nanoseconds per operation**.

Size	Operation	ArrayList	LinkedList	ArrayDeque
10,000	Add/remove at end	54.4	55.1	20.9
	Insert/remove at front	2,089	27.2	9.3
	Insert/remove in middle	212	14,662	n/a
	`get(k)` random k	14.0	5,725	n/a
	Iterate, per element	26.2*	49.7*	42.7*
100,000	Add/remove at end	12.1	18.3	2.5
	Insert/remove at front	9,441	6.0	1.6
	Insert/remove in middle	4,353	231,568	n/a
	`get(k)` random k	9.3	72,656	n/a
	Iterate, per element	1.32	3.93	1.43
1,000,000	Add/remove at end	3.2	8.3	28.5†
	Insert/remove at front	133,383	5.4	2.1
	Insert/remove in middle	46,830	2,325,453	n/a
	`get(k)` random k	8.3	1,300,943	n/a
	Iterate, per element	1.26	4.51	1.23

* the 10,000-element iteration is dominated by JIT warm-up and is not reliable. † an outlier, most likely resizing and garbage collection during that batch; at the other two sizes ArrayDeque is the fastest at the end.

What the data show: ArrayList front and middle edits grow roughly linearly with the size (front: 2 µs → 9 µs → 133 µs), because the elements after the position are shifted with one `System.arraycopy`; LinkedList `get(k)` and middle insertion also grow linearly (5.7 µs → 73 µs → 1.3 ms) because the position must be found by walking nodes, and each node is a separate heap object, so iteration is also 3–4× slower than ArrayList.

Operating boundary and recommendation

- **ArrayList is a reasonable choice** when the playlist is mostly appended to, read by position and iterated, and edits at the front or in the middle are occasional. A human editing a playlist of a few thousand songs pays about 2 µs for a front insert, far below anything noticeable.
- **I would stop recommending it** when edits at the front or middle are *frequent or batched* on large lists: at 1,000,000 elements a front insert costs about 133 µs, so 10,000 of them take about 1.3 s, while an ArrayDeque does the same in about 21 µs in total.

- LinkedList is never the winner in my data: its O(1) front operations are matched or beaten by ArrayDeque, and its middle operations are only O(1) *after* an iterator is already positioned; finding the position costs O(n) and it needs more memory per element.

Changed condition – the "Up Next" queue

Adds at both ends and removals from the front are exactly the operations where ArrayDeque costs 1.6–9 ns at every size, against 2–133 µs for ArrayList front operations. **Re-evaluated recommendation: keep `ArrayList<Song>` for the playlist (random access, index jumps) and use `ArrayDeque<Song>` for Up Next.** ArrayDeque cannot do `get(k)` or middle insertion, which is acceptable for a queue.

Alternative Concept and Complexity Analysis

Operation	ArrayList	LinkedList	ArrayDeque	Plain array, manual shifting
Add at end	O(1) amortised	O(1)	O(1) amortised	O(1) if spare room, else O(n) copy to a bigger array written by hand
Insert at front	O(n)	O(1)	O(1) amortised	O(n) shift
Insert in middle	O(n)	O(n) to find + O(1) to link	not supported	O(n) shift
`get(k)`	O(1)	O(n)	not supported	O(1)
Iterate	O(n), cache-friendly	O(n), pointer chasing	O(n), cache-friendly	O(n)
Space per element (typical 64-bit JVM with compressed references)	one 4-byte slot, plus up to ~50% unused capacity	one node object (about 24 bytes: header, item, next, prev) plus the item	one 4-byte slot, plus up to ~100% slack after doubling	one 4-byte slot, no slack if the size is exact

The plain array has the same complexity as ArrayList but forces the programmer to write and debug the shifting and resizing; ArrayList *is* that array with the code already written. Space figures are typical values, not measured here.

Evidence: designed experiments for every operation, three sizes, an operating boundary backed by numbers, and a justified answer to the Up Next change.

Q3 – Engineer the hashCode–equals Contract (6 marks)

Tests with predictions made before running

To see the behaviour, `Student` overrides `equals` on `regNo` only; the experiment programs use the same idea.

Test	Scans	Prediction and reason	Observed
T1 – looks correct	The *same* object scanned twice, plus one other student	Size 2. The same object has the same identity hash code, lands in the same bucket and `equals` returns true	**2**
T2 – exposes the bug (the given `main`)	Three *different* objects, two with regNo 192311001	Size 3. Without `hashCode`, each object gets a different identity-based hash, so they usually go to different buckets and `equals` is never even called	**3** (and 3 in 1,000 of 1,000 repeated runs)
T3 – mutation (my own)	After fixing `hashCode`, insert a student with regNo 10 and then change `regNo` to 99	`contains` fails and `remove` fails, size stays 1: the object sits in the bucket of the *old* hash	`contains=false`, `size=1`, `remove=false`, `size=1`
T4 – `TreeSet` (my own)	`TreeSet` with a comparator on `name`; two *different* students named Arun	Size 1: the comparator says "equal", so the second is treated as a duplicate although `equals` says they differ	**1**

Supporting observation: for two scans of the same student, `equals` was `true` but `hashCode` values were different (`hash equal=false`), and `contains(equal object)` returned `false`.

Inquiry

- **Why duplicates appear**: the contract requires *equal objects to have equal hash codes*. The default `hashCode()` is based on object identity, so two equal students have different hashes, are sent to different buckets, and `equals` is never asked.
- **How HashSet finds an element**: it computes `hashCode()`, spreads the bits, picks the bucket `(n-1) & hash`, and inside that bucket (list or tree) compares stored hashes first and calls `equals()` only for entries with the same hash. A lookup therefore needs *both* methods to agree.
- **Mutating a field used in `hashCode()`**: the object stays in the old bucket while lookups go to the new hash's bucket, so `contains` and `remove` fail and the object becomes unreachable (a leak). Use immutable fields (`final`) for keys, as the given class already does.
- **Correction and verification**: hash exactly the fields used in `equals` (here only `regNo`, not `name`).

```
@Override public int hashCode() { return regNo.hashCode(); }   // TODO 1
(Objects.hash(regNo) also works)
```

Complete the Code – `Attendance`

Run	Output
Before TODO 1 (no `hashCode`)	`Unique students: 3`
After TODO 1	`Unique students: 2` (matches the expected output)

The difference is exactly the bug: before the fix, the second scan of student 192311001 was stored as a new element.

Alternative Concept and Complexity Analysis – `TreeSet` with a comparator, or `HashMap<String, Student>`

	`HashSet<Student>`	`TreeSet<Student>` (comparator on `regNo`)	`HashMap<String, Student>` keyed by `regNo`
De-duplicate one scan	O(1) average (needs a correct `hashCode`+`equals`)	O(log n) (needs a comparator consistent with equality)	O(1) average; `String` already satisfies the contract, so `Student` needs no `hashCode`
Space for *n* unique students	Θ(n): a table plus a node per element (a HashSet is backed by a HashMap)	Θ(n): one tree node per element, roughly 40 bytes	Θ(n): a table plus a node per entry
Extra	no ordering	sorted iteration for free	can keep or replace the first scan (`putIfAbsent`)

From my Q4 lookup measurements on 1,000,000 string keys, HashMap took about 109 ns per lookup and TreeMap about 1,532 ns, which agrees with O(1) versus O(log n). **Recommendation**: `HashMap<String, Student>` with `putIfAbsent`, or the corrected `HashSet<Student>`; use the `TreeSet` only if sorted output is required.

Evidence: designed tests with predictions, observed sizes, corrected class, and the explanation of the contract.

Q4 – Choosing a Map for a Pharmacy Inventory (6 marks)

Design reasoning before implementation

The pharmacy needs (1) fast lookup by medicine code, (2) a report in code order and (3) a report in the order medicines were added.

Map	`put` / `get`	Iteration	Natural order	Report it produces for free
`HashMap`	O(1) average	O(n + capacity): the table is scanned, including empty buckets	none (order depends on hash codes)	neither
`LinkedHashMap`	O(1) average	O(n): follows an internal linked list of entries	insertion order	insertion-order report
`TreeMap`	O(log n)	O(n), in-order tree walk	sorted by key	code-order report

If the "wrong" map is chosen, extra work is needed: copy the entries to a list and sort – O(n log n) with `HashMap`/`LinkedHashMap` for the code order. For the insertion order, `HashMap` and `TreeMap` do not remember it at all, so an extra insertion counter must be stored with every medicine and used as a sort key (and a sorting cost of O(n log n) per report).

Experiment

Medicines have codes like `MED0123456`. For each size I ran 100,000 random lookups, a full iteration, and both reports (best of 7 repetitions; the build time is a single cold run, so it includes JIT warm-up). For `HashMap`/`TreeMap` the insertion-order report means "copy, then sort by an insertion counter"; for `LinkedHashMap` the code-order report means "copy, then sort by code".

Size	Structure	Build	100,000 lookups (per lookup)	Full iteration	Code-order report	Insertion-order report	Natural order
1,000	HashMap	0.2 ms	2.23 ms (22 ns)	0.04 ms	0.71 ms	8.51 ms*	neither
	LinkedHashMap	0.3 ms	2.40 ms (24 ns)	0.02 ms	0.38 ms	0.02 ms	insertion
	TreeMap	1.4 ms	25.63 ms (256 ns)	0.05 ms	0.11 ms	0.70 ms	code
100,000	HashMap	30.3 ms	4.86 ms (49 ns)	1.98 ms	59.79 ms	139.97 ms	neither
	LinkedHashMap	42.6 ms	9.70 ms (97 ns)	1.53 ms	94.52 ms	3.76 ms	insertion
	TreeMap	104.6 ms	62.10 ms (621 ns)	5.11 ms	3.60 ms	128.94 ms	code
1,000,000	HashMap	388.0 ms	10.92 ms (109 ns)	42.77 ms	714.87 ms	3,023.84 ms	neither
	LinkedHashMap	158.5 ms	12.71 ms (127 ns)	10.14 ms	1,207.18 ms	129.33 ms	insertion
	TreeMap	1,600.4 ms	153.24 ms (1,532 ns)	97.55 ms	74.30 ms	2,999.36 ms	code

* the smallest size is dominated by JIT warm-up; use the larger sizes for conclusions. I checked the output order programmatically: only `TreeMap` iterated in code order and only `LinkedHashMap` iterated in insertion order; `HashMap` matched neither requirement.

The measurements match the predictions: HashMap and LinkedHashMap lookups are about 10× faster than TreeMap at 1,000,000 entries (109–127 ns against 1,532 ns, O(1) against O(log n)); each map

produces its own natural report almost free (3.6–74 ms for TreeMap's sorted report, 3.8–129 ms for LinkedHashMap's insertion report) while the other report costs 60 ms–3 s.

Scalability inquiry

- **Acceptable response time** (my definition): one lookup must finish in **under 1 ms**, which leaves 99% of a 100 ms interactive delay budget for the screen and the database; the nightly reports must finish in under **5 s** as an unattended batch job. All three maps meet both criteria even at 1,000,000 medicines (worst lookup 1.5 µs; worst report 3.0 s), and a hospital pharmacy realistically holds tens of thousands of medicines, not millions.
- **Deployment decision: `LinkedHashMap<String, Medicine>`.** Lookups happen hundreds of times a day (about 100 ns each), the insertion-order report is free, and the code-order report is one `sort` per evening (about 95 ms at 100,000 medicines).
- **Where the remaining cost comes from**: the code-order report – an O(n log n) sort of String keys (about 95 ms of the evening's total at 100,000, 1.2 s at 1,000,000), plus cache misses in lookups (hash → entry object → key String → compare), which is why a lookup is ~100 ns rather than a few ns.
- **Why `Hashtable` is not preferred**: it is a legacy class whose every method is `synchronized` (one coarse lock, slow when threads compete); it forbids `null` keys and values; it has no insertion or sorted order and no modern API (`putIfAbsent`, `compute`, streams). New code uses `HashMap` (single thread) or `ConcurrentHashMap` (several threads).

Alternative Concept and Complexity Analysis – sorted array with `Arrays.binarySearch`

Keep `Medicine[]` sorted by code (plus a parallel `String[] codes` for searching); look up with `Arrays.binarySearch(codes, code)`.

	HashMap	TreeMap	Sorted array + binarySearch
Lookup	O(1) avg: 109 ns at 1M	O(log n): 1,532 ns at 1M	O(log n): 1,340 ns at 1M (24.6 ms / 100,000 lookups at 1,000 items)
Insert one medicine	O(1) avg	O(log n)	**O(n)**: the elements after the position are shifted – 3.9 µs at 100,000, 44.1 µs at 1,000,000
Code-order report	O(n log n) sort	O(n) walk	**O(n)** – already sorted
Space for *n*	Θ(n): table + an entry object (~32 bytes) per medicine	Θ(n): ~40 bytes tree node per medicine	**Θ(n): the smallest** – one 4-byte reference per medicine, no entry objects

The sorted array is compact and gives the code-order report for free, but inserts are O(n) and the insertion-order report is impossible without extra data. It would suit a *static* catalogue that is built once; it is the wrong choice for stock that changes all day.

Evidence: reasoning per map, measured data at three sizes, a stated acceptability criterion and a justified choice.

Q5 – Derive a Generic Ranking Component from a Principle (7 marks)

Design derived from the principle

The principle says a ranking is correct only if its comparison is consistent: transitive, and equal items compare as equal. So the design must make the *comparison* a single well-defined object that is reasoned about once, and keep it separate from the data class.

- **`Comparable<Team>` or `Comparator<Team>`?** `Comparable` gives a class exactly *one* natural order, baked into it. A ranking is a *policy* that changes (rank by points now, by goals scored in a cup, by score for players). So I use separate `Comparator` objects: `Team` stays a plain data class, and the same `Leaderboard` works for players, colleges and quiz teams by passing a different comparator. `Comparable` is right only when one obvious natural order exists (for example a student by register number).
- **Multi-field comparison**: `Comparator.comparingInt(...)` builds one key comparison, `reversed()` flips it (high first), and `thenComparing(...)` is used only when the earlier keys are equal.
- **Generic class and bounded types**: `Leaderboard<T>` takes a `Comparator<? super T>`: a comparator written for a *supertype* can rank any subtype (this is the "consumer super" rule). A bounded variant `NaturalBoard<T extends Comparable<? super T>>` is used when the items rank themselves.
- **Where `? extends T` is useful**: `addAll(Collection<? extends T> src)` accepts a `List<Team>` or a list of any subclass of `T` because it only *reads* from the source ("producer extends").

```
class Leaderboard<T> {
    private final List<T> items = new ArrayList<>();
    private final Comparator<? super T> order;
    Leaderboard(Comparator<? super T> order) { this.order = order; }
    void add(T item) { items.add(item); }
    void addAll(Collection<? extends T> src) { items.addAll(src); }      // wildcard:
producer extends
    List<T> ranking() { List<T> copy = new ArrayList<>(items); copy.sort(order);
return copy; }
}
class NaturalBoard<T extends Comparable<? super T>> extends Leaderboard<T> {   //
bounded type
    NaturalBoard() { super(Comparator.naturalOrder()); }
}
```

I compiled and ran this: a `NaturalBoard<Integer>` ranked `[3,1,2]` as `[1, 2, 3]`, and a `Leaderboard<String>` accepted a `Comparator<Object>`.

Complete the Code

```
List<T> ranking() {                                    // TODO 1
    List<T> copy = new ArrayList<>(items);             // a copy, so items is not
modified
```

```
    copy.sort(order);
    return copy;
}

// TODO 2 (inside main)
Comparator<Team> byRank = Comparator.comparingInt((Team t) -> t.points).reversed()
        .thenComparing(Comparator.comparingInt((Team t) -> t.goalDiff).reversed())
        .thenComparing(t -> t.name);
```

Actual output (matches the expected output): `[Eagles(9,6), Bisons(9,4), Panthers(9,4), Cobras(6,1)]`

Why Bisons is above Panthers: both have 9 points and goal difference 4, so the first two keys are equal; the third key is the name, and "Bisons" comes before "Panthers" alphabetically.

Note the comparator `.reversed()` is applied to the *single* key it follows. Writing `Comparator.comparingInt(...).thenComparing(...).reversed()` would reverse *everything*, including the name order – a common mistake.

Tests

Data set 1 – ties at every level (Panthers 9/4, Eagles 9/6, Cobras 6/1, Bisons 9/4, Hawks 6/1, Ants 6/3, Zebras 9/6 – points tie, goal-difference tie and name decide): `[Eagles(9,6), Zebras(9,6), Bisons(9,4), Panthers(9,4), Ants(6,3), Cobras(6,1), Hawks(6,1)]` – correct, and identical whichever order the input is given.

Data set 2 – built to break faulty comparators

Faulty comparator	Test	Observed
Compares points by subtraction: `(x, y) -> y.points - x.points`	Points −2,000,000,000 (Low), +2,000,000,000 (High), 5 (Mid)	Result `[Low, High, Mid]`; the correct comparator gives `[High, Mid, Low]`. `compare(High, Low)` returned **+294,967,296** instead of a negative number: the subtraction overflowed `int`
Ignores the name tie-breaker	The same 7 teams, input in forward order, then reversed	Forward: Panthers before Bisons. Reversed input: Zebras before Eagles and Hawks before Cobras. The ranking depends on the input order, so it is not a function of the teams

A brute-force check of my correct comparator on 40 teams with many ties (all pairs for antisymmetry and all triples for transitivity) found **0 violations**.

Correctness inquiry – why my comparator satisfies the principle

Write each team as the tuple (−points, −goalDiff, name). Each component is compared with a total order (`Integer.compare` on ints, `String.compareTo` on names), and `reversed()` merely negates the sign. The comparator is the lexicographic combination of these.

- **Equal items compare as equal**: the result is 0 only if all three components are equal, i.e. the teams are indistinguishable for ranking.

- **Antisymmetry**: the first component where `a` and `b` differ decides the result; swapping the arguments flips the sign of that component, so `compare(a,b) = –compare(b,a)`.
- **Transitivity**: suppose a ranks above b (first difference at component *i*) and b above c (first difference at component *j*). Let *m* = min(*i*, *j*). Before *m* all three teams agree. At *m*: if *i* < *j* then $a_m < b_m = c_m$; if *j* < *i* then $a_m = b_m < c_m$; if *i* = *j* then $a_m < b_m < c_m$. In every case a ranks above c.

The faulty subtraction comparator fails because `y.points - x.points` is not always the true sign of the difference (overflow), and the no-name comparator fails the "equal items" rule: it says two *different* teams are equal, so a stable sort simply keeps the input order.

Alternative Concept and Complexity Analysis

- **`Comparable<Team>`**: `compareTo` would contain the same three-key chain; the ranking is fixed inside `Team`, so ranking players or colleges needs each class to implement its own, and a second ranking (e.g. by goal difference) needs a `Comparator` anyway.
- **Three stable sorts** (by name, then goal difference, then points): correct only because `List.sort` is stable (a merge-sort variant); each pass sorts all *n* elements.

	One chained comparator	`Comparable`	Three stable sorts
Number of O(n log n) sorts	1	1	**3**
Comparisons per element pair	up to 3 key checks, usually 1–2	same	1 per sort
Extra space	O(n) merge buffer	O(n)	O(n) buffer, reused for each sort
Reuse for players, colleges	**Best**: supply different key extractors	Needs the class changed	Needs repeating the order of passes, easy to get wrong

Measured on 500,000 teams: one chained sort **396 ms**, three stable sorts **623 ms**, and both gave the same result. The chain is both faster and easier to reuse.

Evidence: derived design, generic class, tests with ties and a deliberately breaking data set, `List.sort` evidence and a reasoned argument.

Q6 – Priority, Order and Fairness in a Queue (8 marks)

Design

- **Why a plain `Queue` is not enough**: `LinkedList`/`ArrayDeque` serve strictly first-in first-out, so a critical patient (severity 1) who arrives behind 50 minor ones would wait for all of them.
- **`PriorityQueue` with a comparator**: it always keeps the smallest element (by the comparator) at the head. The comparator is: severity ascending, then arrival ascending.
- **Preserving arrival order among equal severity**: `PriorityQueue` is *not* stable – equal elements come out in an unspecified order. The `Patient` therefore carries an `arrival` number taken

from a counter that increases by one at every `admit`, and the comparator uses it as the second key. A timestamp would not do, because two patients can share the same millisecond; the counter is unique.

- **Removing and displaying**: `poll()` removes and returns the next patient (null if empty). To *display* the waiting list in treatment order, copy the queue into a list and sort the copy (or poll from a copy): O(n log n), without changing the real queue.

Complete the Code – `Triage`

```
private final PriorityQueue<Patient> queue = new PriorityQueue<>(            // TODO 1
        Comparator.comparingInt((Patient p) -> p.severity).thenComparingLong(p -> p.arrival));
```

Actual output (matches the expected output): `Meena(S1) Kumar(S1) Lata(S2) Ravi(S3) John(S3) Asha(S5)`

How the counter keeps equal severities in arrival order: Meena (counter 1) and Kumar (counter 4) are both S1; the comparator finds severities equal and then compares 1 < 4, so Meena is served first. The same happens for Ravi (0) before John (2).

Correctness check on a hand-worked instance (8 patients)

Arrival order and severity: Anil 3, Bala 5, Chitra 1, Dev 2, Esha 3, Farah 1, Gopal 4, Hari 2.

By hand: severity 1 → Chitra (arrived 3rd), Farah (6th); severity 2 → Dev (4th), Hari (8th); severity 3 → Anil (1st), Esha (5th); severity 4 → Gopal; severity 5 → Bala.

	Order
Hand-derived	Chitra(S1) Farah(S1) Dev(S2) Hari(S2) Anil(S3) Esha(S3) Gopal(S4) Bala(S5)
Program (repeated `poll`)	Chitra(S1) Farah(S1) Dev(S2) Hari(S2) Anil(S3) Esha(S3) Gopal(S4) Bala(S5) — identical
`System.out.println(queue)`	[Chitra(S1), Dev(S2), Farah(S1), Hari(S2), Esha(S3), Anil(S3), Gopal(S4), Bala(S5)]

Why printing the PriorityQueue directly is not the treatment order: the queue is a binary heap stored in an array. A heap guarantees only that each parent is not larger than its children, so only the head (index 0) is guaranteed to be the next patient. `toString()` and the iterator walk the array in heap layout – in the printout, Dev(S2) appears before Farah(S1).

Growth investigation

Prediction: `add` is O(log n) in the worst case (sift-up) but cheap on average for random keys, because the new element usually stops near the bottom; `poll` is O(log n) (the last element is moved to the root and sifted down through about $\log_2 n$ levels). Both should grow very slowly with *n*.

Measured (ns per operation, best of 5 repetitions of 20,000 operations; sizes 1,000 to 1,000,000):

Queue size	PriorityQueue add	PriorityQueue poll	5 FIFO queues add / serve	Sorted LinkedList add / serve
1,000	91*	1,092*	36 / 27*	9,924 / 115*
10,000	54	424	14 / 10	26,863 / 4
100,000	33	289	17 / 10	265,895 / 5
1,000,000	21	251	14 / 4	3,457,174 / 6

* the smallest size is dominated by JIT warm-up. From 10,000 to 1,000,000 patients (×100) `poll` stayed in the same 250–420 ns band; $\log_2 n$ only grows from 13 to 20 (×1.5), which is within the noise and the effect of caching, so the data are consistent with O(log n) but cannot *prove* the slope. The sorted `LinkedList` add grows about 10× for every 10× in size – clearly O(n).

Inquiry – starvation and a fair rule

A severity-5 patient is served only when no more severe patient is waiting. If severe cases keep arriving faster than they are served, that moment never comes. My simulation (one S5 patient, then two new S1–S3 patients and one served per tick): with the plain priority queue the S5 patient was **still waiting after 10,000 ticks**.

Fair rule: set a maximum waiting time *W*. If the oldest waiting patient has waited more than *W*, that patient goes next; otherwise serve by severity. With *W* = 100 the same S5 patient was served at **tick 101**. A safer clinical variant is to *promote* a patient by one severity level for each *W* waited, never beyond level 2, so that no long-waiting minor case can overtake a critical one; a hard deadline as implemented in Q7 can do that, and the hospital's clinicians must choose the values.

Deployment decision

Yes, the design can serve 5,000 patients a day. A cold, single run of 5,000 admits and 5,000 serves took 1.38 ms in total; even at one million waiting patients each operation stayed below about 300 ns. The queue will never be the bottleneck; the real limits are the clinicians, a thread-safe version (e.g. `PriorityBlockingQueue`) if several desks admit patients, and persistence if the server restarts.

Alternative Concept and Complexity Analysis

Design	Admit	Serve next	Space	Notes
`PriorityQueue` (used)	O(log n)	O(log n)	Θ(n) (array of references)	General, and stays correct if levels are added
`LinkedList` kept sorted	**O(n)**: find the position, then insert (265,895 ns at 100,000)	O(1): `removeFirst`	Θ(n) plus ~24 bytes per node	Fast serve, slow admit

Design	Admit	Serve next	Space	Notes
Array of 5 FIFO queues (one per level)	O(1)	O(1): check at most 5 queues	Θ(n) plus 5 queue objects	Fastest in my data (14 ns / 4 ns at 1,000,000) because the severity range is tiny and fixed; it does not generalise to many levels

Honest conclusion: for exactly five severity levels the array of FIFO queues is the best fit; the `PriorityQueue` is the better general component and is fast enough, which is why I used it.

Evidence: student-designed priority queue, hand-verified correctness check, predicted and measured costs, an explanation of starvation, a fair rule and a deployment decision.

Q7 – Day-4 Design Challenge: Diagnose, Redesign and Validate (13 marks)

Solution selected: the Q6 triage queue, because the requirement changed (the fairness rule) and the original design cannot express it. This is not a swap of one collection for another: the redesign is a different *structure of two views over the same patients*.

Code used for the comparison: `Plain` is the Q6 design; `NaiveAging` is the Q6 design extended in the obvious way (scan for the oldest patient on every call); `TwoIndex` is the redesign. All three share `Patient` (with a unique sequence number `seq` used as the final tie-break key).

A. Diagnose the bottleneck (2 marks)

The Q6 `PriorityQueue` is fast (about 250 ns per `poll` at 1,000,000 patients) but it orders by *one* key fixed at admission and only exposes its head. The fairness rule needs the *oldest* waiting patient, which is not the head. The obvious extension therefore does this on every serve:

```
P oldest = null;
for (P p : pq) if (oldest == null || p.seq < oldest.seq) oldest = p;     // O(n) scan
if (oldest != null && now - oldest.arr > W) { pq.remove(oldest); return oldest; }
// remove(Object) is O(n) too
return pq.poll();
```

Measured evidence for this `NaiveAging` version (2,000 steps of "admit one, serve one" on a preloaded queue):

Waiting patients n	Time per admit + serve	Elements visited per serve
1,000	5,470 ns	1,001
10,000	33,795 ns	10,001
100,000	266,467 ns	100,001

- **Where the time goes**: almost all of it is the scan. Elements visited per serve = n + 1.
- **Which operation grows**: the serve operation; time grows about 6–8× for every 10× in *n* (the rest is a constant cost) – close to linear, instead of the logarithmic cost of the plain queue.
- **Repeated work**: yes – the same unchanged waiting list is re-scanned in full on every call, although only one patient was added or removed since the last call.

- **Characteristic of the class that causes it**: a `PriorityQueue` keeps only the *head* ordered. Iteration and `remove(Object)` have no index and cost O(n), and it can sort by only one comparator at a time.

B. Design objective (2 marks)

The redesign must:

1. keep the fairness rule (an overdue patient is served first, oldest first; otherwise by severity then arrival);
2. make the cost of one serve independent of the number of waiting patients: a measurable target of **under 1 µs per admit + serve at n = 100,000** (at least 100× faster than the 266 µs above) and **O(1) elements visited per serve** instead of n + 1;
3. give exactly the same service order as the naive version (correctness preserved).

C. Revised solution (4 marks)

What changed: the same `Patient` objects are kept in **two** structures: the existing `PriorityQueue` ordered by (severity, `seq`), and a new `ArrayDeque` in arrival order. `admit` adds the patient to both.

Property exploited: patients arrive in order, so the arrival-order deque always has the *oldest* waiting patient at its head – readable in O(1), no scan needed. Nothing has to be sorted by arrival; the order already exists.

Lazy deletion: when a patient is served through one structure, a `served` flag is set; the other structure discards the stale entry when it reaches it at its head. Removing from the middle of a `PriorityQueue` (O(n)) is therefore never needed. Every patient enters each structure once and leaves each once, so the stale entries skipped over all operations total at most one per admit: **O(log n) amortised** per operation.

```
ALGORITHM Admit(p)
  pq.add(p);  byArrival.addLast(p)

ALGORITHM Next(now)
  WHILE byArrival not empty AND head(byArrival).served DO remove head          //
discard stale
  IF byArrival not empty AND now - head(byArrival).arrival > W THEN
      p <- remove head;  p.served <- TRUE;  RETURN p                           //
fairness rule
  WHILE pq not empty DO
      p <- pq.poll()
      IF NOT p.served THEN  p.served <- TRUE;  RETURN p                        //
normal rule
  RETURN NONE
```

```
TriageDesk ---has---> PriorityQueue<Patient>   ordered by (severity, seq)
           ---has---> ArrayDeque<Patient>      arrival order (head = oldest)
Patient: name, severity, arrival, seq, served
```

Operations avoided or reduced: the O(n) scan per serve and the O(n) `PriorityQueue.remove(Object)`.

Extra memory/preprocessing: one extra reference per waiting patient in the deque (about 4 bytes each, an estimate; not measured) plus stale entries (at most one per patient). No preprocessing.

New trade-offs: (1) two structures must stay consistent, so they may be changed only through `admit` and `next`; (2) `Patient` becomes mutable (`served`); (3) the structure is not thread-safe; (4) a hard deadline *W* lets a long-waiting severity-5 patient go ahead of a severity-1 patient, which is clinically questionable – the safer variants are a different *W* per severity level or promoting a patient one level per *W* waited.

D. Prediction before implementing (1 mark)

- `NaiveAging` per call is O(n): its time should grow ×10 when *n* grows ×10, and it should visit n + 1 elements per serve.
- `TwoIndex` should be almost flat (O(log n) amortised): less than 2× growth over a 100× increase of *n*, and visit about 1 element per serve.
- Expected improvement: more than 100× in time per admit + serve at n = 100,000, with **identical** service order.
- The S5 patient who never gets served in the plain design should be served at about tick *W* + 1.

E. Implementation and validation (2 marks)

Same conditions for both: preload *n* patients, then 2,000 steps of admit one and serve one; best of 3 repetitions; *W* set so large that the timed steps measure only the check, not the overdue branch (the overdue branch is validated separately below).

n	NaiveAging (original)	TwoIndex (redesign)	Speed-up	Elements visited per serve
1,000	5,470 ns	211 ns	about 26×	1,001 → 1
10,000	33,795 ns	210 ns	about 161×	10,001 → 1
100,000	266,467 ns	351 ns	about 759×	100,001 → 1

Correctness validation

- *Equivalence*: 200 random workloads (random arrivals of random severities, *W* = 30, random serving) were run through both versions: **0 of 200** produced a different service order.
- *A bug found on the way*: my first run reported **200 of 200 mismatches**. The cause was ties: several patients admitted in the same tick had equal arrival times, and the naive scan picked an arbitrary one of them as "oldest". Adding a unique sequence number `seq` as the last tie-break key to both the comparator and the "oldest" test fixed it. Lesson: a comparison that can tie is not a deterministic rule.
- *Starvation*: plain queue – the S5 patient was still waiting after 10,000 ticks; `TwoIndex` with *W* = 100 – served at tick 101, as predicted.

F. Defence of the final design (2 marks)

- **Was the prediction supported?** Yes. The redesign was nearly flat (211 → 351 ns for ×100 patients; the small rise is probably caching, not an algorithmic change), visited 1 element per serve, and the order was identical.
- **Magnitude**: about 26×, 161× and 759× faster at 1,000, 10,000 and 100,000 waiting patients; the objective (under 1 µs and over 100× at 100,000) was met.
- **New costs**: extra memory for the second index and stale entries, extra code that must keep both structures consistent, a mutable flag, and the clinical risk of a hard deadline (above).
- **When the original is still preferable**: if no fairness rule is needed, the plain `PriorityQueue` is simpler, has one structure and stays at O(log n). Even with the rule, the naive scan is acceptable when the waiting list is small – it costs about 5.5 µs at 1,000 waiting patients, which would be invisible for a hospital that admits 5,000 patients a day and has a few hundred waiting at any moment. The redesign matters when the same component is reused for much larger queues (the brief says it will be), which is why I recommend it as the general solution.

Limits of my evidence: single machine, best-of-3 timing; the timed runs do not exercise the overdue branch (it was validated by the equivalence and starvation tests); memory was estimated, not measured.